

Stage 1 Report

学号：2022010776

姓名：张瀚宸

实验内容

step 1

step1中代码都已经提供，我根据文档阅读了源码，了解了本编译器的大致逻辑和架构。
并且我根据文档中的指示阅读了ply的官方文档的部分章节，大致了解了ply的用法和原理。

step 2

这一步骤的改动不算大，由于文法已经定义，所以只需要让AST和后续步骤支持这一节要求的新的三个Unary操作符即可。

修改主要发生在生成三地址码的过程中和生成汇编的过程中。

生成三地址码的时候需要将新要求的三个表达式加入到对应的Enum之中，并且在对应的visit函数的映射表中加入这三个新的操作符。

与此同时，为了能将新的操作符打印出来，同时也为了后续操作方便，

我将 `Unary.__str__` 中关于操作符名称的部分由原来的判断直接改为打印 `self.op.name`，避免每次重复修改这一部分。

生成汇编时的改动就是在三地址码的基础上支持对应的操作。

根据文档的提示和我自己观察gcc的编译结果，得知 `neg`，`seqz`，`not` 三个操作刚好符合要求。

因此我将其加入到了对应的Enum以及映射表中。

step 3

这一步骤与上一步骤十分类似，需要修改的也是生成三地址码和生成汇编的步骤。

只需要仿照上一步将对应的操作加入到对应的Enum和映射表之中就可以完成了。

step 4

这一步骤与之前不同的是riscv只有 `slt` 指令用于完成 `<`，不能在一步之内完成 `>` `>=` `<=` `==` `!=` 操作。

`and` `or` 同理。

因此这一步不能单纯扩展映射表来完成操作，而是需要更多的 `elif` 分支来手动完成逻辑。

`and` `or` 的实现方式文档已经给出了提示，我按照提示完成了代码编写，即手动添加多条指令到

`self.seq` 中。

对于比较操作，我的实现方法如下：

- `>`：反向小于
- `>=`：非小于
- `<=`：非大于
- `==`：通过 `xor` 指令并判断是否为零
- `!=`：通过 `xor` 指令并判断是否非零

在判断是否为零的时候我借助了 `sltiu` 指令。

这一指令需要一个立即数，因此我在 `utils/reg.py` 中实现了 `Imm` 类代表立即数。

`Imm` 类继承自 `Reg` 类来避免在寄存器分配步骤中为立即数分配寄存器。

即利用 `backend/reg/bruteregalloc.py:83`和`:90` 的判断跳过寄存器分配函数。

思考题

step 1

1. 可以编译。因为目前`namer`和`typer`只对`program`进行检查不进行更改，且此程序符合检查的所有要求，因此可以编译。
2. 如果没有`return`语句，则会在 `frontend/tacgen/tacgen.py:113` 中被自动添加一个，并在 `backend/riscv/riscvasmmemitter.py:70` 中被将返回值默认置为零。如果有`return`语句但没有具体的返回值，则有悖于 `ply_parser.py` 中定义的文法，会在此处报`Syntax error`。
3. 三种运算符服务于三个不同的步骤，第一个服务于生成抽象语法树的过程，其值代表的是 `Minidecaf`语法中规定的一元运算符。
第二个服务于生成三地址码过程，其值代表的是三地址码中的一元运算符。
第三个服务于生成汇编的过程，其值代表的是RISC-V汇编中的一元运算汇编指令。

step 2

1. `-(~ 2147483647)`会发生越界。因为`~2147483647`的结果是`-2147483648`，而`-(2147483648)`会发生越界。

step 3

1. 左操作数取`-2147483648`，右操作数取`-1`。在x86-64 Ubuntu上运行结果为`Floating point exception`，在`qemu-riscv32`中运行结果为`-2147483648`，在x86-64 Windows上程序卡死。

step 4

1. 首先，在效率上短路求值可以避免无用运算，因此可以将耗时操作放在尽量靠后的位置，极大提升程序运行效率。
同时，利用短路求值特性可以做一些高级操作（不仅局限于C），比如`flag and func()`可以当做条件执行，`something or "default"`可以用来当做`something`为空时的默认值。